INSTRUCTOR REFERENCE GUIDE

Full-Stack Deployment & Project Architecture

Complete step-by-step master guide for teaching and demonstrating full-stack app creation from Mega Prompt to Live Cloud Deployment.

Pipeline Architecture:

- **Frontend UI:** React + Vite + Tailwind CSS (Hosted on **Vercel**)
- **Backend API:** Node.js / Express Server (Hosted on **Render**)
- **Database & Auth:** PostgreSQL + User Authentication (Hosted on **Supabase**)
- **AI Engine:** Google Gemini API / Groq API
- **Version Control:** Git & GitHub Auto-Deployment

1 Step 1: The Mega Prompt (Instructional Blueprint)

When starting an AI-assisted project from scratch, always use a single, highly structured **Mega Prompt**. This prevents miscommunication, missed files, and structural mistakes.

5 Core Pillars of a Great Mega Prompt:

1. **Role & Goal:** Specify exact project domain (e.g., "Full-stack Crop Advisory AI system").
2. **Tech Stack:** Frontend (React + Vite + Tailwind), Backend (Express), DB (Supabase), AI (Gemini).
3. **Folder Blueprint:** Explicitly require `client/` and `server/` separation with root `.gitignore` and `database.sql`.
4. **Authentication & Security:** Require Supabase Auth, secure CORS handling, and strict environment isolation.
5. **Complete API & Feature Contract:** List every route (`/api/farms`, `/api/advisory`, `/health`).

Sample Mega Prompt Template for Teaching:

```
"You are an expert full-stack developer. Create a production-ready application called <AppName>.
Architecture:
1. Root directory contains /client (Vite+React), /server (Node+Express), database.sql, .env.example, and
.gitignore.
2. Database: PostgreSQL schema for Supabase with users & business tables + Row Level Security (RLS).
3. Backend: Express server with CORS allowing localhost:5173 and *.vercel.app, rate limiting, and Gemini
API integration.
4. Frontend: Modern responsive Tailwind UI with clean auth flows and dashboard."
```

2 Step 2: Database & Auth Setup (Supabase)

Supabase provides the managed PostgreSQL database, authentication, and security policies.

Step-by-Step Dashboard Setup:

1. Log in to **supabase.com** and click **New Project**.
2. Choose a project name, database password, and your nearest geographical region.
3. Once provisioned, go to **Project Settings** → **API**.

Required Credentials:

Credential Name	Where to Copy From	Where It Belongs	Security Level
Project URL	Project Settings → API → Project URL	client/.env & server/.env	Public
anon Public Key	Project Settings → API → Project API Keys	client/.env (VITE_SUPABASE_ANON_KEY)	Safe for browser
service_role Key	Project Settings → API → Project API Keys	server/.env only	**CRITICAL SECRET (Never leak)**

Executing Database SQL:

Navigate to the **SQL Editor** tab on Supabase, paste your project's `database.sql` file, and click **Run**. Ensure user authentication is enabled under **Authentication** → **Providers** → **Email**.

3 Step 3: Obtaining Gemini API Key & Secret Management

Obtaining Google Gemini API Key:

1. Visit **aistudio.google.com**.
2. Sign in using your Google developer account.
3. Click **Get API Key** in the left navigation sidebar.
4. Click **Create API Key** (choose a new Google Cloud project or use default).
5. Copy the string starting with `AQ...` or `AIzaSy...`.

> **Rule 1 of Web Development:** Never commit your real API keys or `.env` files to GitHub. GitHub scans every push and will reject commits with live keys.

Correct .gitignore Configuration:

```
# Put this in root .gitignore
node_modules/
.env
.env.local
client/.env
server/.env
dist/
build/
```

Safe vs Unsafe Environment Configuration:

File	Purpose	Committed to Git?
`.env.example`	Template showing required key names (values replaced with dummy placeholders)	**YES (Safe)**
`client/.env`	Local frontend keys (VITE_SUPABASE_URL, VITE_API_BASE_URL)	**NO (Ignored)**
`server/.env`	Local backend secrets (GEMINI_API_KEY, SERVICE_ROLE_KEY)	**NO (Ignored)**

4 Step 4: Local Testing & Git Version Control

Running Locally for Validation:

Open two separate terminal windows inside your project folder:

Terminal 1: Server

```
cd server
npm install
npm run dev
# Runs on port 5000
```

Terminal 2: Client

```
cd client
npm install
npm run dev
# Runs on http://localhost:5173
```

Git Master Cheat Sheet (Initializing & Pushing to GitHub):

```
# 1. Initialize git repository in root folder
git init

# 2. Stage all files (verifying .gitignore is active)
git add .

# 3. Create your first commit
git commit -m "Initial commit: Project full stack"

# 4. Ensure branch is called 'main'
git branch -M main

# 5. Link to your GitHub repository
git remote add origin https://github.com/<USERNAME>/<REPO_NAME>.git

# 6. Push to GitHub
git push -u origin main
```

> **Subsequent Updates:** Whenever you change code, simply run:
> `git add . && git commit -m "update message" && git push origin main`

5 Step 5: Backend Deployment (Render)

Render provides free hosting for long-running Node/Express web servers.

Deployment Instructions:

1. Log into **render.com** and click **New +** → **Web Service**.
2. Select **Build and deploy from a Git repository** and connect your GitHub repo.
3. Fill out the exact build configuration parameters shown below:

Render Field	Required Value	Explanation
Name	`cropai-backend` (or your custom name)	Will form your public URL
Root Directory	`server`	Tells Render the backend code is in `/server`
Runtime	`Node`	Node.js environment
Build Command	`npm install`	Installs backend dependencies
Start Command	`node index.js`	Starts the express server
Instance Type	`Free`	Free tier

Render Environment Variables (Add under "Environment"):

Key	Value
`PORT`	5000
`SUPABASE_URL`	https://your-project.supabase.co
`SUPABASE_SERVICE_ROLE_KEY`	eyJ... (from Supabase API settings)
`GEMINI_API_KEY`	Your Gemini Studio API Key
`FRONTEND_URL`	https://your-frontend.vercel.app (optional, due to wildcard CORS)

6 Step 6: Frontend Deployment (Vercel) & Final Audit

Vercel Configuration:

1. Log into **vercel.com** → **Add New...** → **Project**.
2. Select your GitHub repository.
3. Set **Root Directory** to: `client` (Click "Edit" next to Root Directory).
4. Ensure **Framework Preset** is set to `Vite`.
5. Expand **Environment Variables** and enter the 3 client keys:

Variable Name	Value Description
`VITE_SUPABASE_URL`	https://your-project.supabase.co
`VITE_SUPABASE_ANON_KEY`	Supabase anon public key
`VITE_API_BASE_URL`	**https://<render-app-name>.onrender.com/api**

> **Notice:** Vercel and Render connect directly via `VITE_API_BASE_URL`! There is no separate pairing dashboard needed.

Master Testing & Demonstration Checklist:

- [ ] Verify Backend Health: Visit `https://<app>.onrender.com/health` → Expect `{"status":"ok"}`
- [ ] Verify Frontend: Visit your live `https://<app>.vercel.app` domain
- [ ] Test User Auth: Create a new account with email & password
- [ ] Test Database CRUD: Create/edit an item and ensure it persists on reload
- [ ] Test AI Integration: Request AI advisory/prediction and inspect response
- [ ] Browser Console Check: Press F12 → Confirm 0 CORS or Network errors